ChatGPT

NumPy 배열 결합 함수들 쉬운 비교 가이드

Executive summary

- NumPy 배열 결합은 크게 **기존 축을 따라 이어붙이기(`concatenate` 계열)** 와 **새 축을 만들어 쌓기(`stack` 계열)** 로 나뉜다. [1]
- `vstack/hstack/column_stack`는 자주 쓰는 패턴을 **짧게** 만들기 위한 편의 함수이며, 특히 **1D가 2D로 어떻게 바뀌는지**가 핵심 차이이다. [2]
- `r_[ ]/c_[ ]`는 "인덱싱 트릭"이라 **빠르고 편하지만 자동 변환 규칙이 많아** 실무 코드에선 가독성에 주의해야 한다. [3]
- `row_stack`은 `vstack`**의 별칭(alias)** 로서 **deprecated 되었고**, 이후 릴리스 노트에서 `vstack`**으로 대체되어 제거됨**이 명시되어 있다. 따라서 새 코드는 `vstack`이 안전하다. [4]

상세 설명과 코드 예제

축(axis)·차원(ndim) 감 잡기

- 2D 배열에서 `axis=0`은 **행(위아래) 방향**, `axis=1`은 **열(좌우) 방향**이다.
- `np.concatenate`는 **기존 축(axis)** 길이를 늘리는 방식(차원 수는 보통 유지)이다. [5]
- `np.stack`은 **새 축**을 하나 만들어 결과 차원을 1 늘리는 방식이다. [6]

```
flowchart LR
  A["A: (2, 3)"] -->|concatenate axis=0 / vstack| C["(4, 3)  # 행 늘어남"]
  A -->|concatenate axis=1 / hstack| D["(2, 6)  # 열 늘어남"]
  A -->|stack axis=0| E["(K, 2, 3)  # 새 축(배치) 추가"]
  A -->|stack axis=-1| F["(2, 3, K)  # 마지막에 새 축"]
```

np.concatenate

- 정의: 여러 배열을 **기존 축(axis)** 을 따라 이어붙이는 가장 범용적인 결합 함수이다. [5]
- 입력 차원/axis 동작: 입력 배열은 **같은 차원 수**여야 하며, **붙일 축(axis)을 제외한 나머지 shape는 같아야** 한다. 기본 `axis=0`이다. [5]
- 반환 shape 규칙: 결과 shape는 "axis 방향 길이만 합쳐지고" 나머지 축은 그대로 유지된다. `axis=None`이면 **모두 펼쳐서(flatten) 1D로 결합**한다. [5]
- 주요 차이점/주의점: `vstack/hstack`보다 **일반적이고 예측 가능**하다. 반대로 axis를 헷갈리면 의도와 다른 방향으로 붙기 쉽다. [5]
- 실무 권장: 3D 이상 텐서에서도 축을 명시해야 할 때 1순위이다. [5]

```python
import numpy as np

a = np.array([[1, 2],
              [3, 4]])        # (2, 2)
b = np.array([[5, 6]])        # (1, 2)

c0 = np.concatenate((a, b), axis=0)    # 행 방향 결합
```

```
print("axis=0:", c0.shape, "\n", c0)

c1 = np.concatenate((a, b.T), axis=1) # 열 방향 결합 (b.T 필요)
print("\naxis=1:", c1.shape, "\n", c1)

cflat = np.concatenate((a, b), axis=None)  # 모두 펼쳐 1D로 결합
print("\naxis=None:", cflat.shape, cflat)
```

np.stack

- 정의: 여러 배열을 **새 축(new axis)** 으로 쌓아 묶는 함수이다. [6]
- 입력 차원/axis 동작: 입력 배열은 **모두 같은 shape**여야 한다. `axis`는 "새 축이 들어갈 위치"이며 기본 `axis=0`이다. [6]
- 반환 shape 규칙: 결과는 입력보다 **차원이 1 더 많다**. [6]
- 주요 차이점/주의점: `concatenate`는 "길이를 늘리고", `stack`은 "차원을 하나 추가"한다는 점이 완전히 다르다. [7]
- 실무 권장: 배치(batch), 시간(time), 실험 반복(trial) 축처럼 "묶음 축"이 필요할 때 적합하다. [6]

```
import numpy as np

x = np.array([[1, 2, 3],
              [4, 5, 6]])    # (2, 3)
y = x + 10                   # (2, 3)

s0 = np.stack([x, y], axis=0)     # (2, 2, 3)
print("stack axis=0:", s0.shape)

slast = np.stack([x, y], axis=-1)  # (2, 3, 2)
print("stack axis=-1:", slast.shape)
```

np.vstack

- 정의: 배열을 **세로(위아래)** 로 쌓는 편의 함수이다. [8]
- 입력 차원/axis 동작: **1D `(N,)`는 먼저 `(1, N)`으로 바꾼 뒤**, 첫 번째 축(axis=0)으로 결합하는 것과 같다. [8]
- 반환 shape 규칙: 결과는 **최소 2D**이다. [8]
- 주요 차이점/주의점: "1D도 그냥 길게 이어붙이겠지"라고 생각하면 오해가 생긴다(실제로는 2D가 된다). 1D를 그냥 이어붙이고 싶으면 `hstack`(1D) 또는 `concatenate(axis=0/None)`가 더 자연스럽다. [9]
- 실무 권장: (샘플, 피처) 형태의 2D 테이블에서 **샘플(행) 추가**에 적합하다. [8]

```
import numpy as np

a = np.array([1, 2, 3])
b = np.array([4, 5, 6])
v1 = np.vstack((a, b))
print("1D vstack:", v1.shape, "\n", v1)

c = np.array([[10],
              [20]])
```

```
d = np.array([[30]])
v2 = np.vstack((c, d))
print("\n2D vstack:", v2.shape, "\n", v2)
```

np.hstack

- 정의: 배열을 **가로(좌우)** 로 쌓는 편의 함수이다. [10]
- 입력 차원/axis 동작: 기본은 **axis=1(두 번째 축) 결합**이지만, **1D 배열일 때만 예외적으로 axis=0 결합**이다. [10]
- 반환 shape 규칙: 1D 입력이면 결과도 1D가 된다. 2D 이상이면 열 방향이 합쳐진다. [10]
- 주요 차이점/주의점: 1D에서 `hstack`은 "열 결합"이 아니라 "그냥 이어붙이기"다. 1D 벡터들을 열로 만들 목적이면 `column_stack` / `c_[ ]` / `reshape(-1,1)`을 먼저 고려하는 것이 안전하다. [11]
- 실무 권장: 2D 테이블에서 **피처(열) 추가**에 적합하다. [10]

```
import numpy as np

a = np.array([1, 2, 3])
b = np.array([10, 20])
h1 = np.hstack((a, b))
print("1D hstack:", h1.shape, h1)

x = np.array([[1],
              [2],
              [3]])   # (3,1)
y = np.array([[4],
              [5],
              [6]])   # (3,1)
h2 = np.hstack((x, y))
print("\n2D hstack:", h2.shape, "\n", h2)
```

np.column_stack

- 정의: 여러 1D 배열을 **열(column)** 로 세워 2D로 만들거나, 2D 배열들을 `hstack`처럼 붙이는 함수이다. [12]
- 입력 차원/axis 동작: 1D는 먼저 2D "열 벡터"로 바뀌고, 2D는 그대로 `hstack`처럼 붙는다. [12]
- 반환 shape 규칙: 결과는 **항상 2D**이며, 모든 입력의 **첫 번째 차원(행 수)이 같아야** 한다. [12]
- 주요 차이점/주의점: 1D feature들을 열로 묶을 때 가장 직관적이다. 반면 "어느 축으로 붙일지"를 세밀하게 통제해야 하면 `concatenate`가 더 안전하다. [13]
- 실무 권장: `(n_samples, n_features)` 디자인 매트릭스를 만들 때 표준 선택지이다. [12]

```
import numpy as np

f1 = np.array([1, 2, 3])
f2 = np.array([10, 20, 30])
f3 = np.array([100, 200, 300])

X = np.column_stack((f1, f2, f3))
print("column_stack 1D -> 2D:", X.shape, "\n", X)
```

```
A = np.array([[1, 2],
              [3, 4]])      # (2,2)
B = np.array([[5],
              [6]])         # (2,1)
H = np.column_stack((A, B))
print("\ncolumn_stack 2D:", H.shape, "\n", H)
```

np.row_stack

- 정의(역사적 동작): 배열을 세로로 쌓는 함수로 문서화되어 있으며, 1D `(N,)`를 `(1,N)`으로 바꾼 뒤 axis=0로 결합해 결과가 최소 2D가 되도록 한다. [14]
- 최신 NumPy에서의 상태: `np.row_stack`은 `vstack`**의 별칭(alias)** 로서 deprecated 되었고, 이후 릴리스 노트에서 `vstack`**으로 대체되어 제거됨**이 명시되어 있다. 새 코드는 `vstack`이 안전하다. [4]
- 주요 차이점/주의점: 환경에 따라 `np.row_stack` 호출이 경고/오류로 이어질 수 있으니, 실무에선 `np.vstack`으로 통일하는 편이 좋다. [15]
- 실무 권장: 레거시 코드의 `row_stack`은 "검색/치환으로 `vstack` 정리"가 가장 현실적인 처방이다. [16]

```
import numpy as np

a = np.array([1, 2, 3])
b = np.array([4, 5, 6])

# 최신 환경에서 row_stack이 없을 수 있으니 안전하게 처리한다
row_stack = getattr(np, "row_stack", np.vstack)

out = row_stack((a, b))
print(out.shape)
print(out)
```

np.c_[ ]

- 정의: "인덱싱 트릭" 객체로, 대괄호 안의 표현을 결합 연산으로 바꿔준다. 문서상 `r_`**의 특정 설정을 줄여쓴 것**이라고 설명되어 있다. [17]
- 입력 차원/axis 동작(핵심만):
- `c_`는 내부 설정상 배열을 **최소 2D로 올리고**, 1D는 **열 벡터**가 되도록 shape에 1을 덧붙인다. [18]
- 이후 결합 축은 설정 문자열의 의미상 **마지막 축(axis=-1)** 쪽으로 동작하는 것이 핵심이며, 2D에서는 이것이 곧 **열 방향(axis=1)** 과 같다. [17]
- 반환 shape 규칙: 1D 두 개를 넣으면 보통 `(N, 2)` 2D가 된다. [18]
- 주요 차이점/주의점: 짧게 쓰기 좋지만 자동 변환이 많다. 특히 3D 이상에서 축을 "두 번째 축"으로 착각하면 의도가 어긋날 수 있으니, 이런 경우는 `concatenate`로 axis를 명시하는 편이 안전하다. [19]
- 실무 권장: 노트북/프로토타입에서 빠르게 컬럼을 붙일 때는 유용하지만, 협업 코드에서는 `column_stack`/`concatenate`가 더 읽기 쉽다. [20]

```
import numpy as np

a = np.array([1, 2, 3])
b = np.array([4, 5, 6])

C = np.c_[a, b]
```

```
print("c_ with 1D:", C.shape, "\n", C)

D = np.c_[a, 0, b]   # 스칼라도 열처럼 들어간다( dtype 변화 가능 )
print("\nc_ with scalar:", D.shape, "\n", D)
```

np.r_[ ]

- 정의: "인덱싱 트릭" 객체로, 대괄호 안의 슬라이스/배열을 **첫 번째 축(axis=0)** 방향 결합 또는 범위 생성으로 바꿔준다. [21]
- 입력 차원/axis 동작:
- `np.r_[a, b]` 처럼 쉼표로 배열이 나열되면 **첫 축으로 결합**한다. [21]
- `np.r_[start:stop:step]` 는 범위를 만들며, step이 허수(예: `5j`)이면 점 개수로 해석되어 `linspace` 처럼 끝을 포함한다. [21]
- 반환 shape 규칙: 범위 생성은 보통 1D, 배열 결합은 입력 형태에 따라 1D~ND가 된다. [21]
- 주요 차이점/주의점: "결합"과 "범위 생성"을 한 문법에 함께 담아서 편하지만, 실무에선 의도가 불분명해질 수 있다. 교육용/실험용이 아니라면 `np.arange` / `np.linspace` / `np.concatenate` 가 더 명확하다. [22]
- 실무 권장: 디버그용 데이터, 간단한 테스트 벡터를 빠르게 만들 때 적합하다. [21]

```
import numpy as np

a = np.array([1, 2, 3])
b = np.array([4, 5, 6])

x = np.r_[a, 0, b]
print("r_ concat:", x.shape, x)

rng = np.r_[0:5]        # 0,1,2,3,4
print("\nr_ range:", rng.shape, rng)

lin = np.r_[0:1:5j]    # linspace처럼 끝 포함 5개 점
print("\nr_ linspace-like:", lin.shape, lin)
```

비교 표

아래 표는 "최신 NumPy 기준"에서 핵심 규칙만 뽑아 정리한 것이다. (`row_stack` 은 deprecated 및 제거 흐름이 명확하므로, 새 코드는 `vstack` 전제로 보는 것이 안전하다.) [4]

함수/연산자	주 입력 차원(대표)	축 기본값/기본 결합 방향	반환 차원(대략)	비고(차이점/주의점)
`vstack`	1D~3D	1D를 `(1,N)` 으로 만든 뒤 axis=0 결합	**최소 2D**	1D가 2D로 바뀐다(자주 헷갈림). [8]
`hstack`	1D~3D	기본 axis=1, **단 1D만 axis=0**	입력 ndim 유지	1D는 "그냥 이어붙이기"다. 열 목적이면 `column_stack` / `c_` / reshape가 낫다. [11]

함수/연산자	주 입력 차원(대표)	축 기본값/기본 결합 방향	반환 차원(대략)	비고(차이점/주의점)
`column_stack`	1D 또는 2D	1D를 열 벡터로 바꾼 뒤 열 방향 결합	**항상 2D**	feature 벡터들을 (샘플,피처)로 만들 때 가장 직관적이다. [12]
`row_stack`	(레거시) 1D~3D	`vstack`과 같은 규칙	**최소 2D**	`vstack`의 alias로 deprecated 되었고 이후 제거됨이 릴리스 노트에 명시되어 있다. [23]
`concatenate`	모든 ndim	`axis=0` 기본, `axis=None`이면 1D로 펼친 뒤 결합	입력 ndim 유지 (단 axis=None은 1D)	가장 범용이며 shape 제약이 명확하다. [5]
`stack`	모든 ndim	새 축 위치 `axis=0` 기본	**입력 ndim + 1**	"배치/시간/실험" 축 생성에 적합하다. [6]
`c_[ ]`	주로 1D~2D	최소 2D로 올린 뒤 주로 열 쪽 결합(2D에선 axis=1과 같은 효과)	대개 ≥2D	빠르지만 자동 변환이 많다. 실무는 명시 API 선호. [24]
`r_[ ]`	1D~ND	axis=0 결합 + 슬라이스는 범위 생성	상황에 따라 1D~ND	결합/범위 생성이 같이 있어 편하지만 혼동 가능성이 있다. [22]

결론과 실무 권장 사례

선택 기준만 기억하면 끝이다

- "축을 늘리지 않고 이어붙이기"면 `concatenate`가 표준 선택이다. axis를 명시하고, 그 축을 제외한 shape를 맞추면 된다. [5]
- "새 축을 추가해 묶기(배치/시간/실험 반복)"면 `stack`이 표준 선택이다. 결과 차원이 1 늘어난다. [6]
- 2D 테이블 기준으로 "행 추가"는 `vstack`, "열 추가"는 `hstack`이 읽기 쉽다. 다만 1D에서는 `hstack`이 axis=0으로 동작하는 예외가 있으니(1D는 그냥 이어붙임), 1D 벡터를 열로 만들 목적이면 `column_stack` / `c_[ ]` /reshape가 더 안전하다. [25]
- `row_stack`은 `vstack`의 alias로 deprecated 되었고 이후 제거되는 흐름이므로, 새 코드는 `vstack`을 쓰는 것이 안전하다. [4]
- `r_[ ]` / `c_[ ]`는 교육·실험·노트북에선 강력하지만, 자동 규칙이 많아 협업 코드에선 `concatenate/stack/column_stack` 같은 명시적 API가 유지보수에 유리하다. [26]

자주 하는 실수와 예방법

- `(N,)`(1D)와 `(N,1)`(2D 열 벡터)을 혼동하면 결합 축이 어긋난다. "열 벡터가 필요하면 `reshape(-1,1)` 또는 `column_stack`"을 규칙으로 두면 실수가 줄어든다. [27]
- `hstack`의 1D 예외(1D는 axis=0 결합)를 모르면 결과 차원이 예상과 달라진다. [10]
- `r_[0:1:5j]`는 편하지만 읽는 사람이 `linspace`를 바로 떠올리기 어렵다. 실무에선 `np.linspace(0, 1, 5)`가 더 명확하다. [21]

확장 질문

- `concatenate`와 `stack`을 조합해 `(time, batch, feature)` 같은 3D 텐서를 설계할 때, 축 설계 원칙을 어떻게 정하는 것이 좋은가.
- 1D 벡터 표준을 `(N,)`, `(1,N)`, `(N,1)` 중 무엇으로 통일하는 것이 데이터 파이프라인에서 가장 안정적인가.
- `c_[ ]/r_[ ]` 같은 단축 문법은 교육에서 어느 단계까지 허용하고, 언제부터 `concatenate/stack`처럼 명시 API로 전환시키는 것이 좋은가.
- 큰 데이터에서 반복적으로 `vstack/hstack`로 배열을 "계속 키우는" 방식이 느려질 수 있는 이유(복사/메모리 관점)는 무엇이며, 이를 피하는 대표 패턴은 무엇인가. [28]

---

[1] [5] numpy.concatenate — NumPy v2.4 Manual
https://numpy.org/doc/stable/reference/generated/numpy.concatenate.html

[2] [8] [9] [25] numpy.vstack — NumPy v2.4 Manual
https://numpy.org/doc/stable/reference/generated/numpy.vstack.html

[3] [21] [22] [26] numpy.r_ — NumPy v2.4 Manual
https://numpy.org/doc/stable/reference/generated/numpy.r_.html

[4] [15] [16] [23] NumPy 2.0.0 Release Notes — NumPy v2.5.dev0 Manual
https://numpy.org/devdocs/release/2.0.0-notes.html

[6] [7] numpy.stack — NumPy v2.4 Manual
https://numpy.org/doc/stable/reference/generated/numpy.stack.html

[10] [11] numpy.hstack — NumPy v2.4 Manual
https://numpy.org/doc/stable/reference/generated/numpy.hstack.html

[12] [13] [20] [27] numpy.column_stack — NumPy v2.4 Manual
https://numpy.org/doc/stable/reference/generated/numpy.column_stack.html

[14] numpy.row_stack — NumPy v1.21 Manual
https://numpy.org/doc/1.21/reference/generated/numpy.row_stack.html

[17] [18] [19] [24] numpy.c_ — NumPy v2.4 Manual
https://numpy.org/doc/stable/reference/generated/numpy.c_.html

[28] Copies and views — NumPy v2.4 Manual
https://numpy.org/doc/stable/user/basics.copies.html?utm_source=chatgpt.com